



Requirements Engineering

Software Engineering

Software Requirements

- ❖ According to the IEEE standard a requirement is a
 - **Condition** or **capability** needed by a user to solve a problem or achieve an objective.
 - **Condition** or **capability** that must be met or possessed by a system or a system component to satisfy a contract, standard, specification, or other formally imposed documents.
 - **Document representation** of a condition or capability as in (1) or (2)
- ❖ This definition emphasizes that requirements can describe either
 - What the system must do and how the system should perform
 - They must be formally documented to ensure clarity and agreement between **stakeholders**.
- ❖ A **stakeholder** is any individual, group, or organization that has an interest or concern in a project, product, or business outcome.

Software Requirements

❖ Types of Stakeholder:

- Internal: People or groups within the organization involved in or affected by the project.
 - E.g Employees, managers, project teams, owners, shareholders.
- External: People or groups outside the organization who have an interest in the project.
 - E.g. Customers, suppliers, regulatory agencies, investors, community groups, competitors.
- Primary: Directly involved with or affected by the project.
 - Examples: End users, clients, project sponsors.
- Secondary: Indirectly affected by the project or its outcomes.
 - Examples: Media, local communities, the general public.

❖ Effectively managing stakeholder relationships, expectations, and communication is critical for achieving project success and alignment with business goals.

Requirements - Different Types

❖ **Functional Requirements**

- Specify the behaviors, functions, or features that a system or component must provide. They define what the system should do.
- Example: "The system shall allow users to log in with a username and password."

❖ **Non-Functional Requirements**

- Specify the quality attributes, system properties, or constraints under which a system must operate. They define how the system should perform, rather than what it does.
- Example: "The system shall be able to handle 1,000 concurrent users with no performance degradation."

❖ **Performance Requirements**

- Specify how well the system performs certain functions, often in terms of speed, efficiency, or resource usage.
- Example: "The system shall load a page within 2 seconds."

Requirements - Different Types

❖ Interface Requirements

- Describe how the system will interact with other systems or components, including software, hardware, and user interfaces.
- Example: "The system shall support communication with external databases via API calls."

❖ Design Requirements

- Are limitations or restrictions placed on the design of the system, often driven by regulatory, technical, or organizational factors.
- Example: "The system shall be built using only open-source software components."

❖ Security Requirements

- Define the necessary measures to protect the system from unauthorized access, misuse, or vulnerabilities.
- Example: "The system shall encrypt all data in transit using SSL/TLS protocols."

Requirements - Different Types

❖ Reliability Requirements

- Describe the expected reliability of the system, including the probability of failure-free operation over a certain period.
- Example: "The system shall have a 99.9% uptime over a year."

❖ Usability Requirements

- Focus on how easy and effective it is for users to learn, operate, and interact with the system.
- Example: "The system shall be accessible to users with disabilities according to [WCAG 2.1](#) standards."

❖ Safety Requirements

- Describe the precautions that must be taken to ensure the safety of users or the environment when interacting with the system.
- Example: "The system shall automatically shut down in the case of overheating."

Requirements - Different Types

❖ Maintainability Requirements

- Define how easily the system can be maintained, including how updates or repairs are managed over time.
- Example: "The system shall be designed so that software updates can be applied without downtime."

❖ Scalability Requirements

- Specify the system's ability to grow and handle increased demand without significant changes to its architecture.
- Example: "The system shall scale to support up to 10,000 users without major changes in architecture."

Requirements - Different Levels

❖ Requirements exist on different levels:

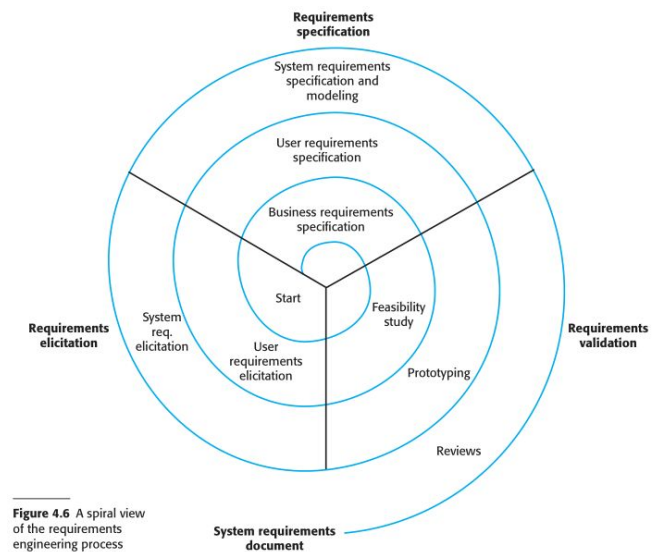
- **Business Requirements:** Describe the top-level objectives, goals, and needs of the organization or customer that develop the software system. They outline why the software is being developed and what problems it will solve for the business or stakeholders.
 - Focus on business context, strategic goals and the "why" behind the system's existence.
 - For examples: Increase sales by 15%, improve customer satisfaction by 20% and etc.
- **Stakeholder Requirements** (sometimes referred to as User Requirements)
 - Represent what stakeholders expect from the system in terms of functionality and quality.
 - Examples:
 - The system shall allow customers to place orders online.
 - The system shall provide administrative staff with reporting capabilities.

Requirements - Different Levels

- ❖ Requirements exist on different levels ...
 - **System (or Software) Requirements:** Describe the detailed functionality, behavior, and constraints of the software system. The system requirements should define exactly what and how the software shall be implemented and tested.
 - A functional example: The system shall generate a report of daily sales.
 - A non-functional example: The system shall respond to user inputs within 2 seconds.
- ❖ Requirements are base for everything and heavily affect architecture, design, and implementation. Massive requirement changes are costly, especially late in the development.

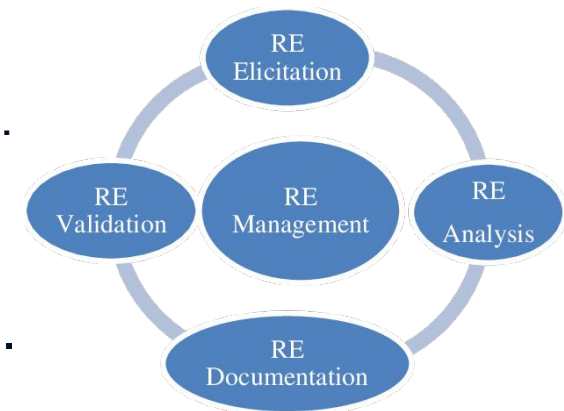
Requirement Engineering

- ❖ It is a systematic and disciplined approach to define, document, and maintain the requirements for a software system.
- ❖ It involves understanding what the stakeholders need from the system and translating the needs into detailed specifications that guide the design, development, and validation of the software.
- ❖ Requirements are developed in an iterative process
- ❖ The main goal of requirement engineering is to ensure that the software system meets the the stakeholders expectations and operates correctly within the defined environment.



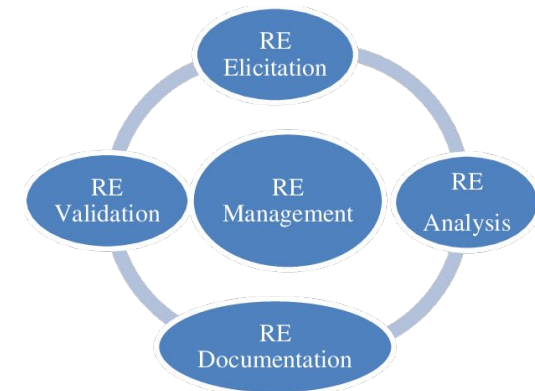
Requirement Engineering - Key Activities

- ❖ **Requirement Elicitation:** The process of gathering requirements from stakeholders through interviews, workshops, observation, prototyping, etc.
 - Understanding user needs, business objectives, and constraints.
- ❖ **Requirements Analysis:** Reviewing and refining the elicited requirements to identify any conflicts, overlaps, or ambiguities.
 - Ensuring the requirements are complete, consistent, and feasible.
- ❖ **Requirements Specification:** Writing a formal document (the Software Requirement Specification, **SRS**) that includes the detailed descriptions of the functional and non-functional requirements in a structured and clear manner.



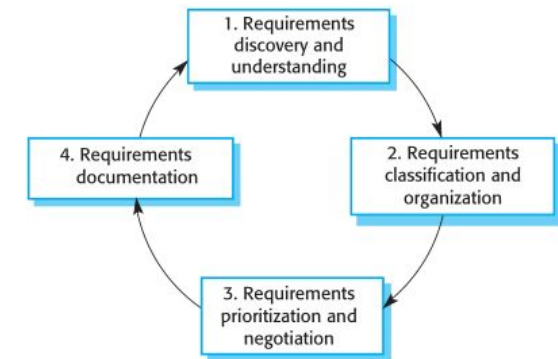
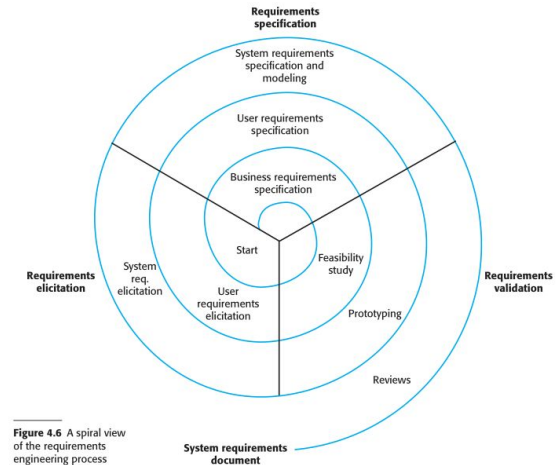
Requirement Engineering - Key Activities

- ❖ **Requirement Validation:** The process of ensuring that the requirements are correct, complete, and aligned with stakeholder expectations.
 - Using techniques like requirements reviews, inspections, prototyping and testing.
- ❖ **Requirements Management:** Requirements often evolve over time and handling changes to requirements throughout the project lifecycle is critical.
 - By tracking changes, maintaining traceability, and ensuring that any new requirements fit within the scope of the project.
- ❖ Generally *Requirement Engineering* emphasizes on a structured approach to ensure that requirements are effectively captured, clearly documented, and correctly developed to reduce risks associated with incomplete or incorrect requirements and to align software development with stakeholder needs.



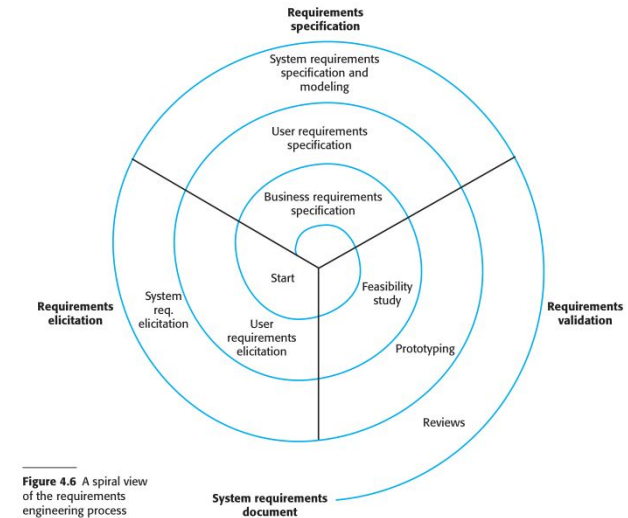
Requirement Engineering - Elicitation & Analysis Activities

- ❖ Requirements discovery and understanding
 - Communicate to stakeholders to discover their requirements
- ❖ Requirements classification and organization
 - The unstructured collection of requirements are grouped and organized in coherent clusters.
- ❖ Requirements prioritization and negotiation
 - When multiple stakeholders are involved, requirements will conflict. This activity is concerned with prioritizing requirements, and finding and resolving requirements conflicts through negotiation
- ❖ Requirements documentation
 - The requirements are documented and input into the next round of the spiral



Requirement Engineering - Requirements Specification

- ❖ The process of writing down the stakeholders and system requirements in requirement documents.
- ❖ Requirements shall be clear, unambiguous, understandable and complete.
- ❖ **Stakeholder requirements** are written in natural language, diagrams and tables.
- ❖ **System requirements** are written in natural language, forms, graphical, or mathematical (e.g. finite-state machines or sets) notations.
 - Mathematical notation is called formal notation. Requirements written using mathematical models are unambiguous and there is no room for different interpretations. But not all understand a formal specification

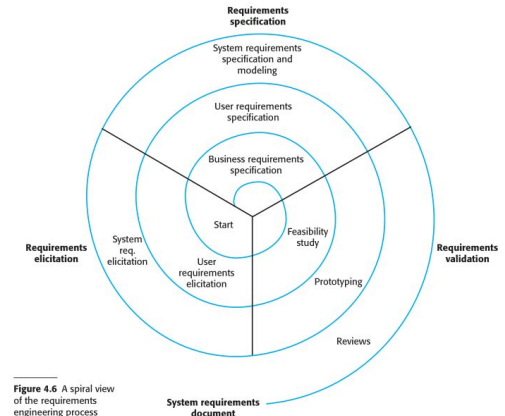


Requirement Engineering - Requirements Specification

- ❖ **Natural Language Notation:** Requirements are written using sentences in a natural language. Each sentence should express a requirement.
 - Mandatory requirements are written using “**shall**”, desirable requirements are not essential and are written using “**should**” and fact statements are written using “**will**”.
- ❖ **Structured Natural Language:** Requirements are written in a standard way rather than as free-form text.
- ❖ **Graphical Notations:** Graphical models, supplemented by text, are used to define functional requirements for a system. **Unified Modeling Language** use case and sequence diagrams are commonly used.
- ❖ **Requirements Validation** is the process of checking that requirements define the system that the stakeholders exactly expect.

Requirement Engineering - Requirements Validation

- ❖ Different types of check should be done:
 - **Consistency**: requirements in the document shall not conflict.
 - **Completeness**: the requirements document should include requirements that define all functions and constraints intended
 - **Realism**: the requirements should be checked to ensure that they can be implemented within the proposed budget for the system.
 - **Verifiability**: To ensure that the delivered system meets all specified requirements.
 - **Validity**: Requirements should be testable. If a test is difficult or impossible to design, it means that it will be difficult to implement and it should be reconsidered.
- ❖ **Software Requirement Specification(SRS)** details all the project's requirements as well as providing an overall description of the product like design, assumptions, and dependencies, project management and etc.



Requirement Engineering - Good Requirement Criteria

- ❖ **Correct:** The requirement shall accurately address and describe an actual need of stakeholders and reflect the actual goals and constraints of the system.
- ❖ **Unambiguous:** The requirement must be expressed clearly with a single, precise interpretation, avoiding any misunderstanding and misinterpretation.
- ❖ **Consistent:** The requirement shall not conflict with other requirements or any relevant standards, and it shall align with the overall system objectives.
- ❖ **Achievable:** The requirement should be realistic and achievable within the constraints of budget, time, technology, and resources available.
- ❖ **Verifiable:** It shall be possible to verify the requirement through testing, inspection, analysis, or demonstration. The requirement must be measurable so that its implementation can be validated.

Requirement Engineering - Good Requirement Criteria

❖ **Complete:** The requirement shall be well-described

- **Stakeholder:** It shall include all necessary information to describe the stakeholder's need.
- **System:** It shall include all aspects necessary for implementation and testing.
- *The requirement shall be brief and to the point, without unnecessary details or repetition.*
- **Embedded systems:** The following information should present:

- Input (or stimuli): What is the input? A specific value or an event?
- Transformation: Modification algorithm
- Output: Is it an event, multiple event or specific values?
- Timing: When should the output be generated, for how long etc.



❖ **Traceable:** The requirement shall be traceable from origin through to implementation and testing, ensuring that all requirements are fulfilled.

- Each requirement shall be assigned a unique identifier (e.g., REQ-001, REQ-002)

Requirement Engineering - Good Requirement Criteria

- ❖ **Prioritized:** Requirements should be prioritized based on their importance and stability. This helps in managing changes and focusing on what is most critical to the system's success.
- ❖ **Atomic:** A system requirement shall be atomic and It shall be detailed enough so that it cannot be broken down into smaller requirements
- ❖ **Examples:**
 - **Bad:** When the system receives an unlock request it shall unlock the doors (**not well-described**)
 - **Good:** When the system receives the `<input_unlock_all_doors>` signal it shall activate the following outputs within 100ms in order to unlock the doors: `<output_unlock_front_left_door>`, `<output_unlock_front_right_door>`, `<output_unlock_rear_left_door>` and `<output_unlock_rear_right_door>`
 - **Bad:** The system shall react to any lock/unlock request immediately (Untestable)
 - **Good:** The system shall fulfill any lock/unlock request within 100ms

Requirement Engineering - A Typical SRS Structure

- ❖ A typical **SRS** document includes the following sections
 - **Introduction:** Background and context of the requirements.
 - **Overall Description:** General overview of the system and its environment.
 - **Requirements:** Detailed functional requirements, non-functional requirements and external Interface requirements; organized hierarchically.
 - **Other Requirements:** E.g. regulatory requirements, maintenance and support requirements and etc.
 - **Appendices:** Provide a glossary of terms used in the document, include any models and define data elements, formats, and structures used in the system.
 - **Index:** An alphabetical index of terms and topics covered in the SRS document for quick reference.

Requirement Engineering

- ❖ A Typical Stakeholder Requirement Structure
 - **ID:** A unique identifier for each requirement (e.g., REQ-001).
 - **Title:** A brief title or name for the requirement.
 - **Description:** A clear statement of the requirement.
 - **Rationale:** (Optional) Why this requirement is needed.
 - **Priority:** (Optional) The importance of the requirement (e.g., high, medium, low).
 - **Source:** The stakeholder or group who provided the requirement.
 - **Dependencies:** Any dependencies on other requirements or systems.
 - **Verification:** How the requirement will be tested or verified. (test, inspection, analysis, demo)
- ❖ In agile, **User Stories** are often used. A user story is a short and simple description of a feature or function from the perspective of an end user.
 - Typically follows the format: "As a [type of user], I want [goal] so that [reason(s)]."

A Stakeholder/User Requirement Example

❖ A Stakeholder Requirement Example

- **ID:** REQ-010
- **Title:** Password reset via a self-service portal.
- **Description:** The system shall allow users to reset their passwords via a self-service portal.
- **Rationale:** To improve user experience and reduce dependency on IT support.
- **Priority:** High
- **Source:** Product Owner, End-users
- **Verification:** Test cases and demonstration
- **Dependencies:** REQ-005 (User authentication module)

❖ A User Story Example: As a shopper, I want to add items to a wishlist so that I can purchase them later.

Requirement Engineering - Break Down Requirements

- ❖ Breaking down requirements is a structured approach to ensure that each requirement is clear, actionable, and testable. It is the process of decomposing a high-level stakeholder requirement into low-level functional and non-functional detailed system requirements.
 - Develop use cases or scenarios that illustrate how the system will fulfill the high-level requirement.
 - Organize requirements in a hierarchical manner, from high-level down to detailed specifications.
 - It is an iterative process. The product owner needs to take a more active role. If you aren't an absolute expert, you can't get it right on the first try.
- ❖ High-Level Stakeholder Requirement: "The system shall be user-friendly."
 - Decomposed Functional Requirements:
 - The system shall have a user interface with a minimal learning curve.
 - The system shall provide user help and documentation accessible within two clicks.
 - Decomposed Non-Functional Requirements:
 - The system shall be usable by users with minimal training.
 - The system shall respond to user inputs within 2 seconds.

Requirement Engineering

❖ A Typical System Requirement Structure

- **ID:** A unique identifier for each requirement (e.g., REQ-001).
- **Description:** A clear statement of the requirement.
- **Rationale** (Optional): Why this requirement is needed.
- **Acceptance Criteria:** Define the specific conditions that shall be satisfied for the system requirement to be accepted as fulfilled.

❖ Stakeholder Requirement Example:

"The system shall allow users to reset their passwords via a self-service portal."

- The high-level stakeholder requirement shall be decomposed into technical and functional system requirements that describe how the system will implement the feature.
- Each system requirement specifies a property or behavior of the system, along with measurable acceptance criteria to ensure the requirement is met.

Requirement Engineering - System Requirements Examples

❖ System Requirement 1 (User Interface):

- **ID:** REQ-001
- **Description:** The system shall provide a "Forgot Password" link on the login page that directs users to the self-service password reset portal.
- **Rationale:** To enable users to initiate the password reset process from the login screen.
- **Acceptance Criteria:** The "Forgot Password" link is visible and functional on the login page in all major browsers (Chrome, Firefox, Safari, Edge).

❖ System Requirement 2 (Password Reset Form):

- **ID:** REQ-002
- **Description:** The password reset portal shall prompt the user to enter his/her registered email address and display appropriate error messages if the email is invalid or not found in the system.
- **Rationale:** To ensure users can only reset passwords for valid accounts.
- **Acceptance Criteria:** The form correctly validates email inputs and provides error messages for invalid or unregistered email addresses.

Requirement Engineering - System Requirements Examples

❖ System Requirement 3 (Email Notification):

- **ID:** REQ-003
- **Description:** The system shall send an email with a secure, time-limited password reset link to the registered email address upon successful submission of the password reset form.
- **Rationale:** To securely notify users and enable them to reset their passwords.
- **Acceptance Criteria:** The system generates a unique reset link that expires after 24 hours, and the email is delivered within 1 minute of form submission.

❖ System Requirement 4 (Password Reset Link Security):

- **ID:** REQ-004
- **Description:** The password reset link must be encrypted & expire after 24 hours or after a single use.
- **Rationale:** To enhance security and prevent unauthorized password resets.
- **Acceptance Criteria:** The link will be invalid after the first use or after 24 hours, whichever comes first.

Requirement Engineering - System Requirements Examples

- ❖ System Requirement 5 (Password Update Functionality):
 - **ID:** REQ-005
 - **Description:** The password reset portal shall allow users to set a new password, which shall meet the system's password complexity rules (e.g., min. 8 characters, one capital letter, one number)
 - **Rationale:** To ensure strong password security.
 - **Acceptance Criteria:** Users can set a password that follows to the complexity rules, and the system enforces these rules without allowing weak passwords.
- ❖ System Requirement 6 (Logging):
 - **ID:** REQ-006
 - **Description:** The system shall log all password reset requests, including timestamps, user identifiers, and whether the reset was successful or failed.
 - **Rationale:** To provide a log for security and troubleshooting purposes.
 - **Acceptance Criteria:** All relevant data are captured and stored securely for at least 90 days.

Requirements Engineering

❖ Some useful links

- [Using the correct terms – Shall, Will, Should](#)
- [Documenting Functional Requirements](#)
- [Documenting Non-Functional Requirements](#)
- [Writing Requirements: Write Functional Requirements](#)
- [Tips for Writing Clear Requirements - 1](#)
- [Tips for Writing Clear Requirements - 2](#)
- [What is a Functional Requirement?](#)
- [Functional and Nonfunctional Requirements](#)
- [Best Practices for Writing Requirements](#)
- [Stories vs. Requirements](#)
- [3 Ways to Write Clearer Requirements](#)